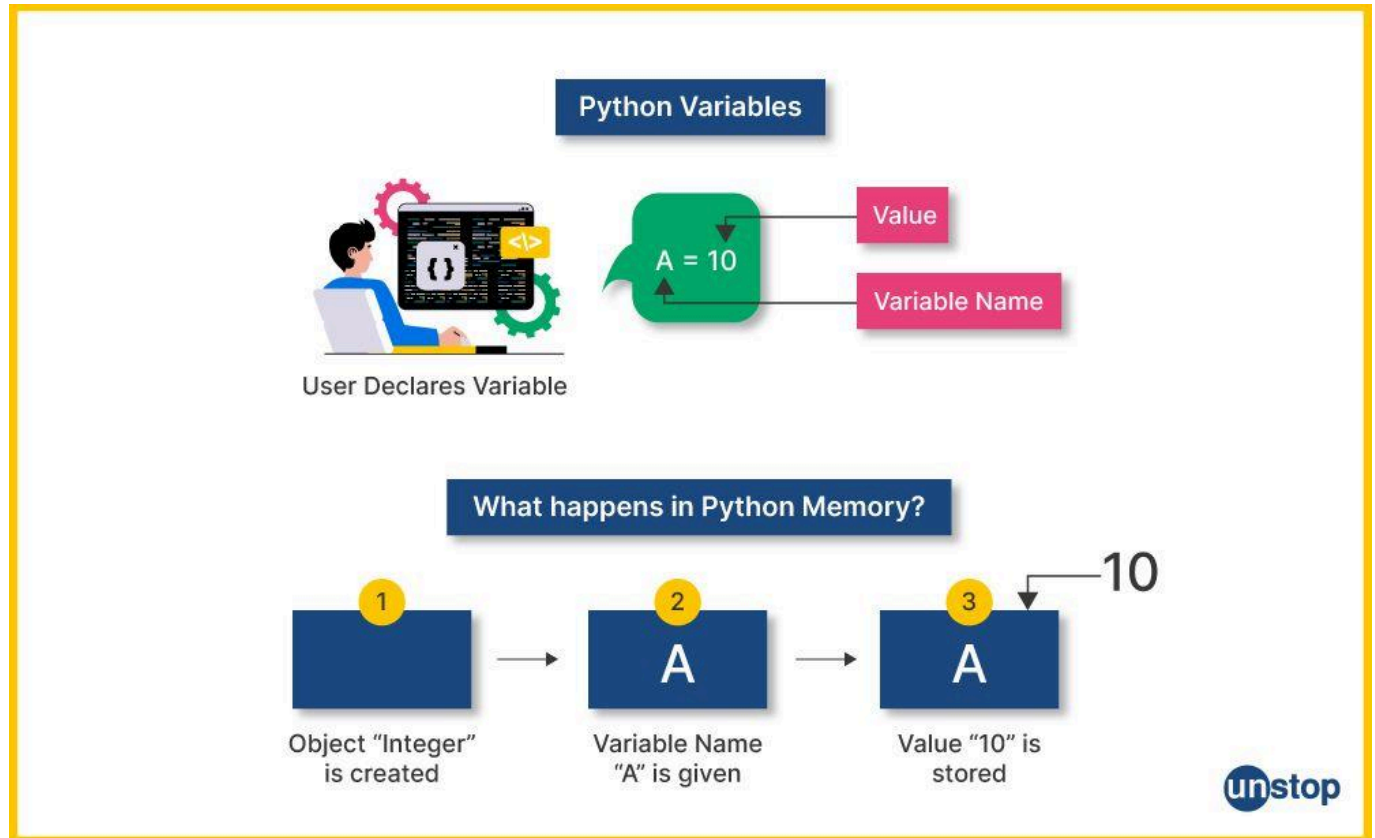
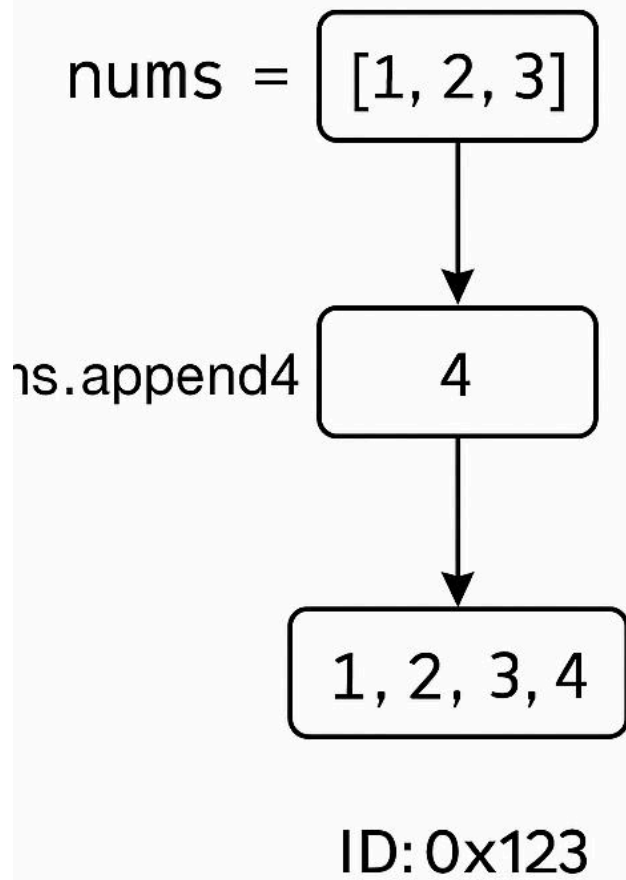


PART 1 — Python Core

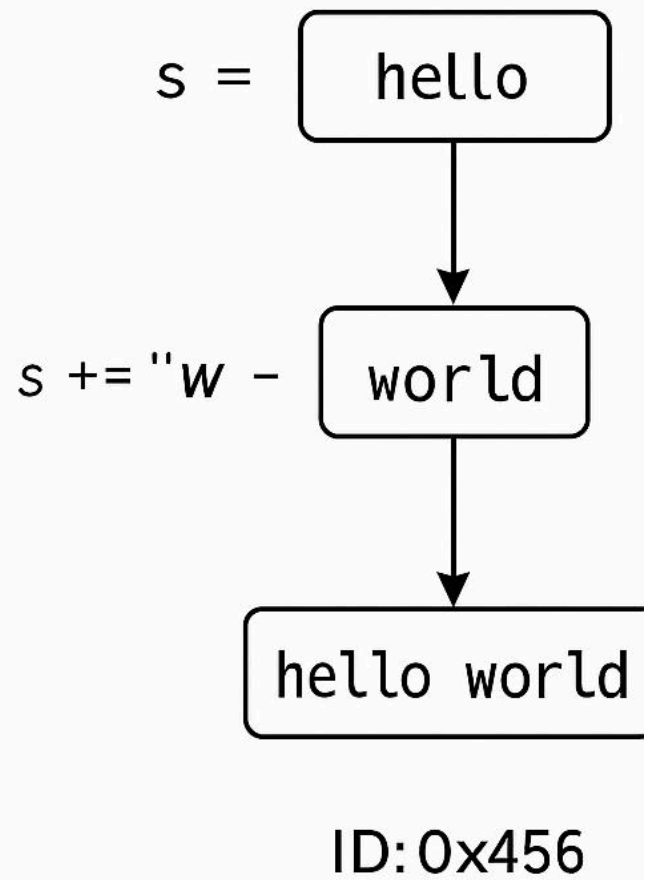
This is the **foundation layer**. If you truly understand these 13 topics, later topics like OOP, iterators, generators, decorators, async programming, and frameworks become much easier.



Mutable



Immutable



Feature	List	Tuple	Set	Dictionary
Symbol	[]	()	{ }	{ k: v }
Sample	[1, 2, "apple"]	(10, 20)	{1, 2, 2, 3} (stores as {1, 2, 3})	{"id": 1, "name": "Jo"}
Concept	Flexible Sequence	Fixed Sequence	Unique Bag	Key-Value Map
Ordered?	✔ Yes	✔ Yes	✗ No	✗ No*
Mutable?	✔ Yes (Changeable)	🔒 No (Immutable)	✔ Yes (Changeable)	✔ Yes (Changeable)
Duplicates?	✔ Allowed	✔ Allowed	🚫 No (Unique only)	🚫 Keys: No ✔ Values: Yes
Best For...	General items	Constants / Configs	Removing duplicates	Fast lookups (IDs)

SLICING IN PYTHON

SYNTAX:

sequence[start:stop:step]

10

20

30

40

50

start Index to begin slice (inclusive)

stop Index to end slice (exclusive)

step Interval between elements

num → | → p

EXAMPLES

my_list[1:4]		[20, 30, 40]
my_list[:3]		[10, 20, 30]
my_list[3:]		[40, 50, 60]
my_list[::2]		[10, 30, 50]

NEGATIVE INDEXING

my_list[-3:-1]

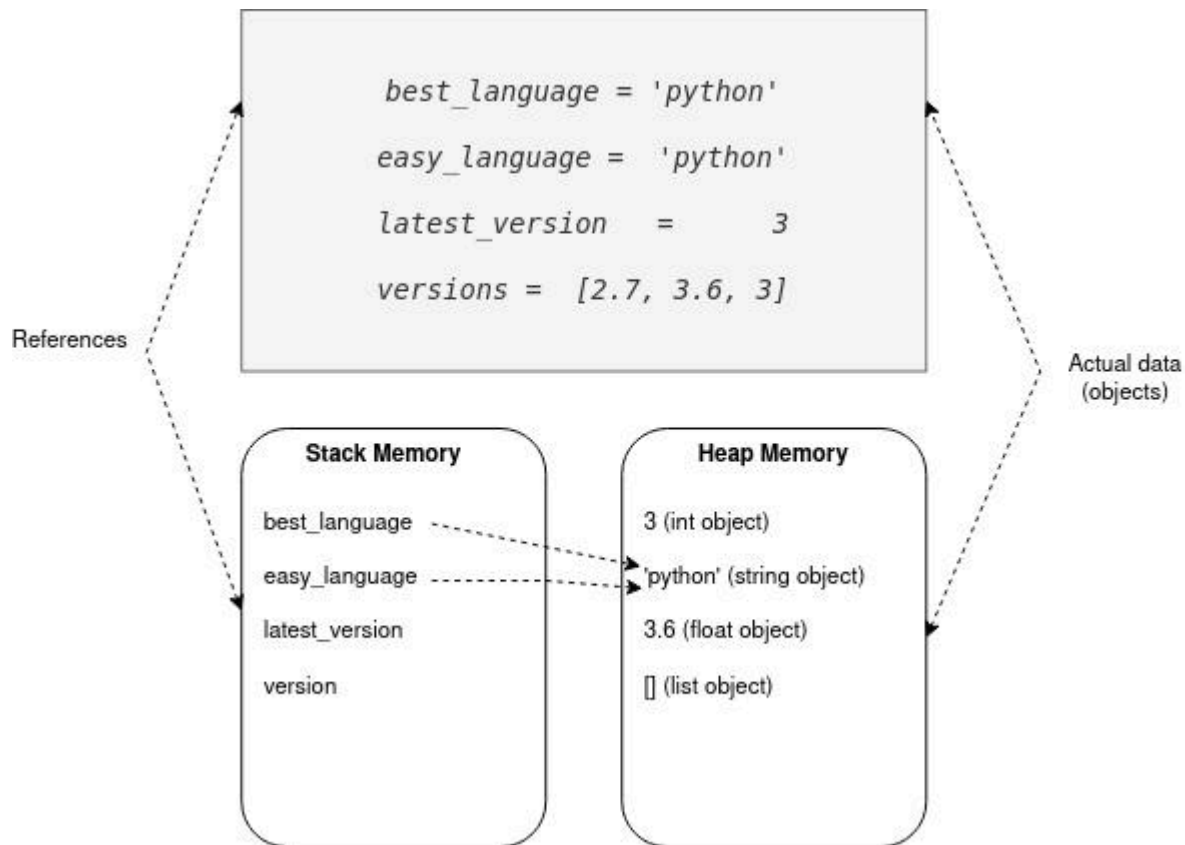
[30, 40
(60, 50, 40, 30, 10)]

SLICING WITH STEP

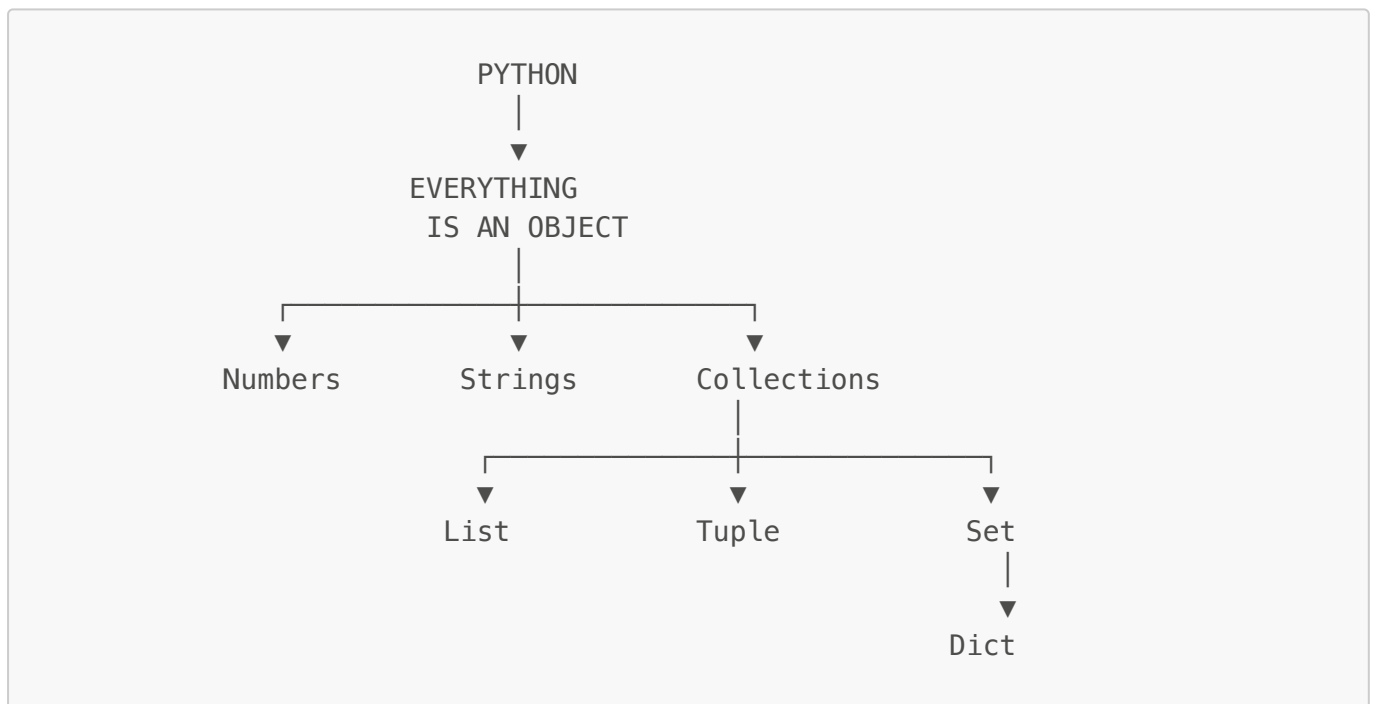
nums[1:6:2]

nums[::-2]

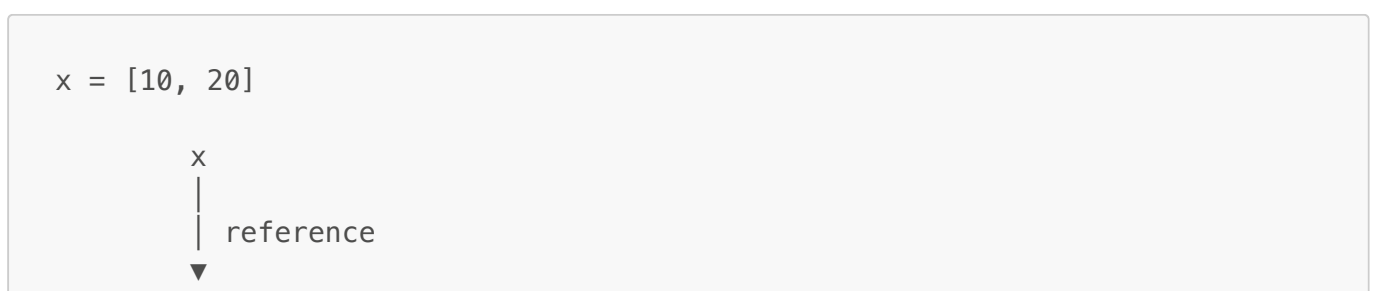
(6, 4, 2, 0)



🧠 First: the BIG mental model



And variables:



[10, 20]

← OBJECT

Very important: `x` is not the object itself. `x` is a **name/reference pointing to an object**.

01. Python Execution Model

When you write:

```
x = 10
print(x)
```

Think:

```
.py source code
  |
  ▼
Python interpreter
  |
  ▼
bytecode
  |
  ▼
Python Virtual Machine
  |
  ▼
execute instructions
```

Conceptually:

```
SOURCE
  |
  ▼
Parsing / compilation
  |
  ▼
BYTECODE
  |
  ▼
Python VM
  |
  ▼
Execution
```

You can inspect bytecode:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

🎯 Mental model

Don't think:

Python directly executes my `.py` file.

Think:

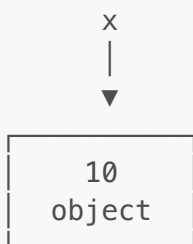
Python turns source code into instructions that the Python runtime executes.

02. Variables & Objects

This is probably the **single most important foundation concept**.

```
x = 10
```

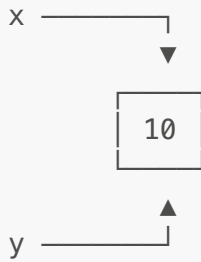
Think:



Now:

```
y = x
```

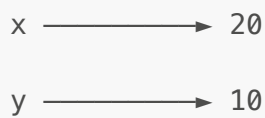
becomes:



There is **one object** and two references.

Now:

```
x = 20
```



Assignment changes the **reference**, not the original object.



Remember

```
variable = reference/name  
object   = actual value/data
```

03. Data Types

Python has many built-in types.

Numbers

```
int  
float  
complex
```

Text

```
str
```

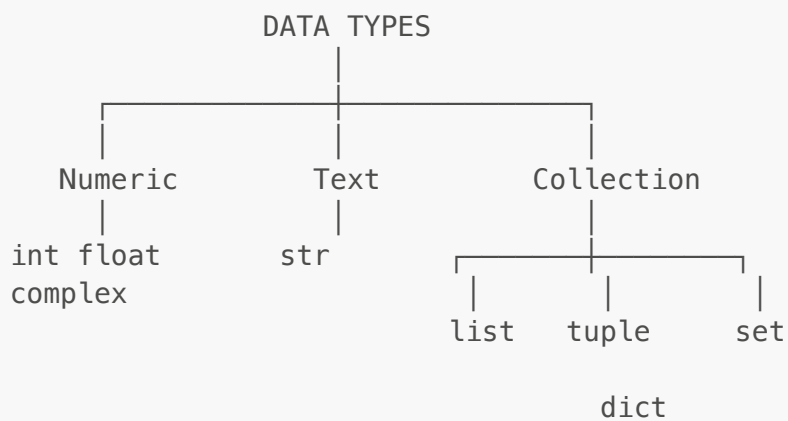
Collections

```
list  
tuple  
set  
dict
```

Other important types

```
bool  
NoneType  
bytes  
bytearray  
range
```

Visual:



You can inspect a type:

```
x = 100  
  
print(type(x))
```

Output:

```
<class 'int'>
```

04. Mutable vs Immutable

This is **extremely important** for interviews and real Python debugging.

Mutable

Can change after creation.

```
list
dict
set
bytearray
```

Example:

```
x = [1, 2, 3]

x.append(4)
```

Same list object changed.

BEFORE		AFTER
[1, 2, 3]	→	[1, 2, 3, 4]
SAME OBJECT		

Immutable

Cannot change after creation.

Examples:

```
int
float
bool
str
tuple
frozenset
bytes
```

Example:

```
x = 10  
x = 20
```

It may look like 10 changed into 20.

But conceptually:

```
x → 10  
    ↓ assignment  
x → 20
```

The 10 object wasn't modified.

★ Golden rule

Mutable = object can change. Immutable = object cannot change.

05. == vs is

One of the most frequently tested Python concepts.

==

Checks:

Do these objects have equal values?

```
a = [1, 2]  
b = [1, 2]  
  
print(a == b)
```

True

Because:

```
[1,2]  
[1,2]
```

same CONTENT

is

Checks:

Are these the exact same object?

```
print(a is b)
```

False

Because:

```
a —————> [1,2]  
b —————> [1,2]
```

Two separate objects.

Visual

==

a → [1,2]

b → [1,2]

CONTENT
SAME

↓

True

is

a → [1,2]

b → [1,2]

DIFFERENT OBJECTS

↓

False

Best practice

For **None**:

```
if x is None:  
    ...
```

Prefer this over:

```
if x == None:  
    ...
```

06. Numbers

Python supports:

```
int  
float  
complex
```

Integer

Python integers can grow very large.

Float

Be aware:

may produce:

because floating-point numbers are represented approximately in binary.

Complex

Operators

13 / 26

```
% modulo  
** exponentiation
```

Example:

```
10 / 3  
# 3.333...  
  
10 // 3  
# 3  
  
10 % 3  
# 1
```

07. Strings

A string is an **immutable sequence of characters**.

```
name = "Python"
```

Think:

```
P   y   t   h   o   n  
0   1   2   3   4   5
```

Therefore:

```
name[0]
```

→ "P"

And:

```
name[-1]
```

→ "n"

Strings support:

```
indexing
slicing
iteration
concatenation
many methods
```

Example:

```
name = "Python"

print(name.upper())
print(name.lower())
print(name[0:3])
```

Output:

```
PYTHON
python
Pyt
```

But:

```
name[0] = "J"
```

✗ Error — strings are immutable.

08. Lists

List = **ordered + mutable + allows duplicates**.

```
numbers = [10, 20, 30]
```

Visual:

```
index:    0    1    2
          ↓    ↓    ↓
list → [10, 20, 30]
```

You can modify:

```
numbers.append(40)
numbers[0] = 99
numbers.pop()
```

Lists can contain different types:

```
data = [
    10,
    "hello",
    3.14,
    True,
    [1, 2]
]
```

Mental model

```
LIST
├── ordered
├── mutable
├── indexed
├── duplicates allowed
└── can contain anything
```

09. Tuples

Tuple = **ordered + immutable**.

```
point = (10, 20)
```

Visual:

```
index:    0    1
          ↓    ↓
tuple → (10, 20)
```

You can read:

```
point[0]
```

But:

```
point[0] = 99
```

✗ Error.

Why use tuples?

Useful for data that shouldn't be modified.

```
coordinates = (22.57, 88.36)
```

Also tuples can be dictionary keys if their contents are hashable:

```
locations = {  
    (22, 88): "Kolkata"  
}
```

10. Sets

Set = **collection of unique elements**.

```
numbers = {1, 2, 3, 3, 3}  
  
print(numbers)
```

Result:

```
{1, 2, 3}
```

Duplicates disappear.

```
{1, 2, 2, 3, 3, 3}  
      |
```

▼
 $\{1, 2, 3\}$

Useful operations:

```
a = {1, 2, 3}
b = {3, 4, 5}
```

Union:

$a \mid b$

$\{1, 2, 3, 4, 5\}$

Intersection:

$a \& b$

$\{3\}$

Difference:

$a - b$

$\{1, 2\}$

Mental model

Set = uniqueness + fast membership operations.

11. Dictionaries

Dictionary stores:

KEY → VALUE

Example:

```
user = {  
    "name": "Alice",  
    "age": 25  
}
```

Visual:

DICTIONARY

"name" → "Alice"

"age" → 25

Access:

```
user["name"]
```

Add:

```
user["city"] = "Kolkata"
```

Update:

```
user["age"] = 26
```

Delete:

```
del user["age"]
```

Important:

```
keys → unique + hashable  
values → can be almost anything
```

Modern Python dictionaries preserve **insertion order**.

12. Slicing

Slicing is:

```
sequence[start : stop : step]
```

The most important rule:

start is included, stop is excluded.

Example:

```
text = "Python"
```

P	y	t	h	o	n
0	1	2	3	4	5

```
text[1:4]
```

gives:

y	t	h
---	---	---

→ "yth"

Visual:

```
text[start : stop : step]  
  |       |       |  
  |       |       └─ jump
```



Examples:

```
text[:3]      # Pyt
text[3:]      # hon
text[::2]     # Pto
text[::-1]    # nohtyP
```

Remember

```
[start : stop)

START → included
STOP  → excluded
```

This same concept applies to:

```
list
tuple
string
range
```

and other slice-supporting objects.

13. Operators

Think of operators as Python's **action symbols**.

Arithmetic

```
+
-
*
/
//
%
**
```

Comparison

```
==  
!=  
>  
<  
>=  
<=
```

Logical

```
and  
or  
not
```

Example:

```
age = 25  
age >= 18 and age < 60
```

Assignment

```
=  
+=  
-=  
*=  
/=
```

Example:

```
x = 10  
x += 5
```

Equivalent conceptually to:

```
x = x + 5
```

Membership

```
in  
not in
```

```
"Py" in "Python"
```

→ True

```
5 in [1, 2, 3]
```

→ False

Identity

```
is  
is not
```

Remember:

```
== → same VALUE?  
is → same OBJECT?
```

Bitwise

```
&  
|  
^  
~  
<<  
>>
```

Example:

```
5 & 3
```

Binary:

```

5 = 101
3 = 011

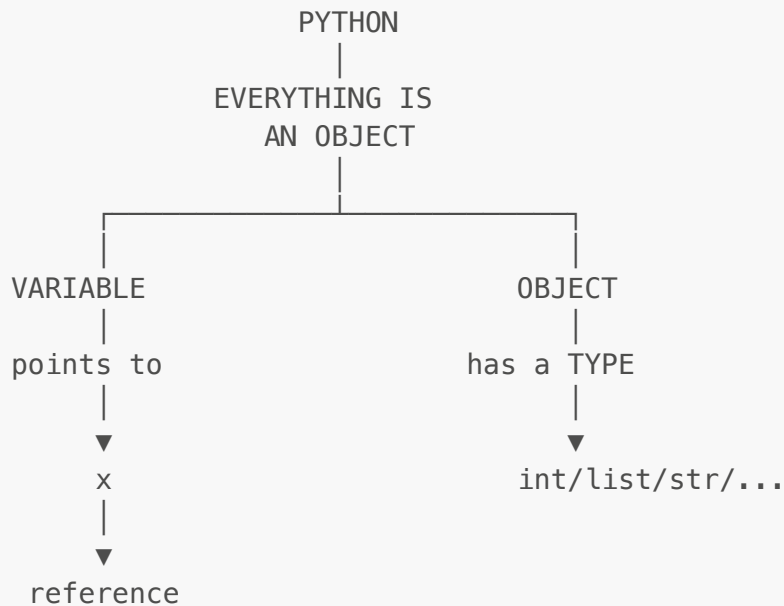
    101
AND 011
----
    001

= 1

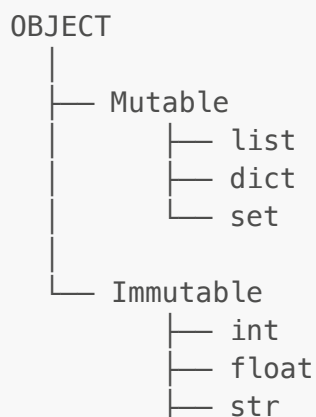
```

FINAL MASTER MAP

If you want to **actually master Part 1**, memorize this structure:

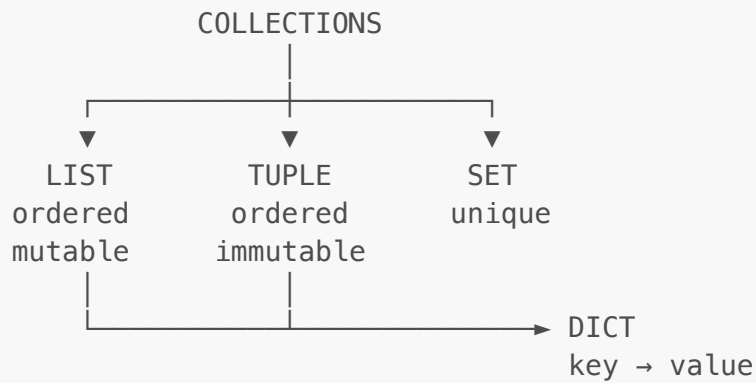


Then:

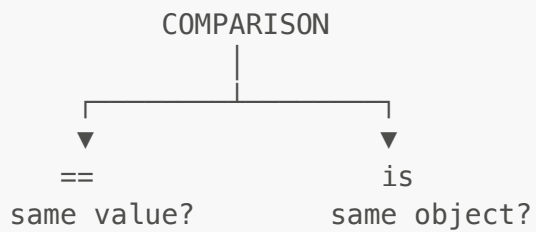


```
├ tuple
└ bool
```

Collections:



Comparison:



Slicing:

```
sequence[start : stop : step]
           |       |
           include  exclude
```

And execution:

```
.py
 |
 ▼
Python runtime
 |
 ▼
bytecode
 |
 ▼
Python VM
 |
```



execution

🔥 The 10 things I would make sure you can explain without looking

1. Why is a variable not the object?
2. What actually happens when `x = y`?
3. What does mutable mean?
4. Why does `a == b` sometimes differ from `a is b`?
5. Why are strings immutable?
6. List vs tuple vs set vs dictionary?
7. Why does `a[1:5]` not include index 5?
8. What is the difference between `/`, `//`, and `%`?
9. What does `in` actually test?
10. What happens conceptually when Python executes a `.py` file?

If you can answer those **from memory and write small examples**, your Python foundation is becoming solid.